Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5
По дисциплине «ОМО»

Выполнил:
Студент 3-го курса
Группы АС-68
Жуков А.А.
Проверил:
Крощенко А.А.

Брест 2026

**Цель работы:** Выполнить моделирование прогнозирующей нелинейной ИНС

**Вариант 2**

| № варианта | a | b | c | d | Кол-во входов ИНС | Кол-во НЭ в скрытом слое |
|---|---|---|---|---|---|---|
| 2 | 0.2 | 0.2 | 0.06 | 0.2 | 8 | 3 |

Для генерации обучающих и тестовых данных использовать функцию:

$$y = a\cos(bx) + c\sin(dx) \ .$$

**Код программы**

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.preprocessing import MinMaxScaler

torch.manual_seed(42)
np.random.seed(42)

# данные
a = 0.2
b = 0.2
c = 0.06
d = 0.2
x = np.arange(0, 150, 0.5)
y = a * np.cos(b * x) + c * np.sin(d * x)

scaler = MinMaxScaler()
y_scaled = scaler.fit_transform(y.reshape(-1, 1)).flatten()

WINDOW_SIZE = 8

def create_dataset(data, window_size):
    X, Y = [], []
    for i in range(len(data) - window_size):
        X.append(data[i:i+window_size])
        Y.append(data[i+window_size])
    return np.array(X), np.array(Y).reshape(-1, 1)

X, Y = create_dataset(y_scaled, WINDOW_SIZE)

train_size = int(len(X) * 0.8)
X_train, X_test = X[:train_size], X[train_size:]
Y_train, Y_test = Y[:train_size], Y[train_size:]
split_idx = train_size + WINDOW_SIZE

X_train_t = torch.tensor(X_train, dtype=torch.float32)
Y_train_t = torch.tensor(Y_train, dtype=torch.float32)
X_test_t = torch.tensor(X_test, dtype=torch.float32)
Y_test_t = torch.tensor(Y_test, dtype=torch.float32)

# архитектура
class RegressionMLP(nn.Module):
```

```
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(WINDOW_SIZE, 3)
        self.sigmoid = nn.Sigmoid()
        self.fc2 = nn.Linear(3, 1)

    def forward(self, x):
        x = self.fc1(x)
        x = self.sigmoid(x)
        x = self.fc2(x)
        return x

model = RegressionMLP()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)

# обучение
epochs = 1000
loss_history = []

for epoch in range(epochs):
    model.train()
    optimizer.zero_grad()
    y_pred = model(X_train_t)
    loss = criterion(y_pred, Y_train_t)
    loss.backward()
    optimizer.step()
    loss_history.append(loss.item())

# оценка
model.eval()
with torch.no_grad():
    train_pred = model(X_train_t).numpy().flatten()
    test_pred = model(X_test_t).numpy().flatten()

Y_train_orig = scaler.inverse_transform(Y_train).flatten()
train_pred_orig = scaler.inverse_transform(train_pred.reshape(-1,
1)).flatten()

Y_test_orig = scaler.inverse_transform(Y_test).flatten()
test_pred_orig = scaler.inverse_transform(test_pred.reshape(-1, 1)).flatten()

# прогнозируемая функция
plt.figure(figsize=(10, 5))

x_train_plot = x[WINDOW_SIZE:split_idx]

plt.plot(x_train_plot, Y_train_orig, color='black', label='Эталон (train)')
plt.plot(x_train_plot, train_pred_orig, color='blue', linestyle='--',
label='Модель (train)')

plt.title("График прогнозируемой функции на участке обучения")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True)
plt.show()

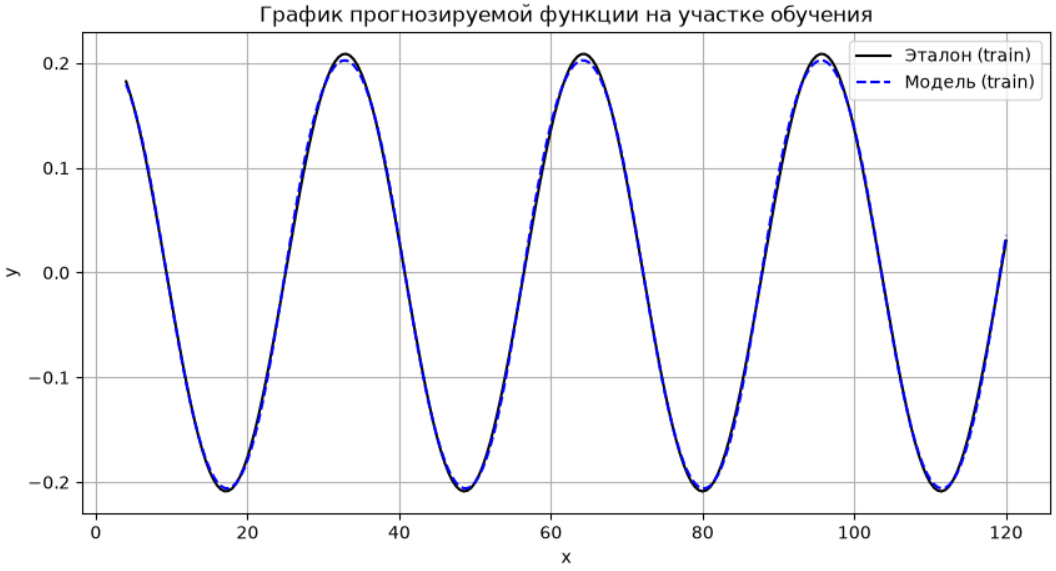
plt.figure(figsize=(10, 5))
plt.plot(loss_history, color='red')
plt.title("График изменения ошибки (MSE) от итерации")
plt.xlabel("Итерация")
plt.ylabel("Ошибка (MSE)")
plt.grid(True)
```

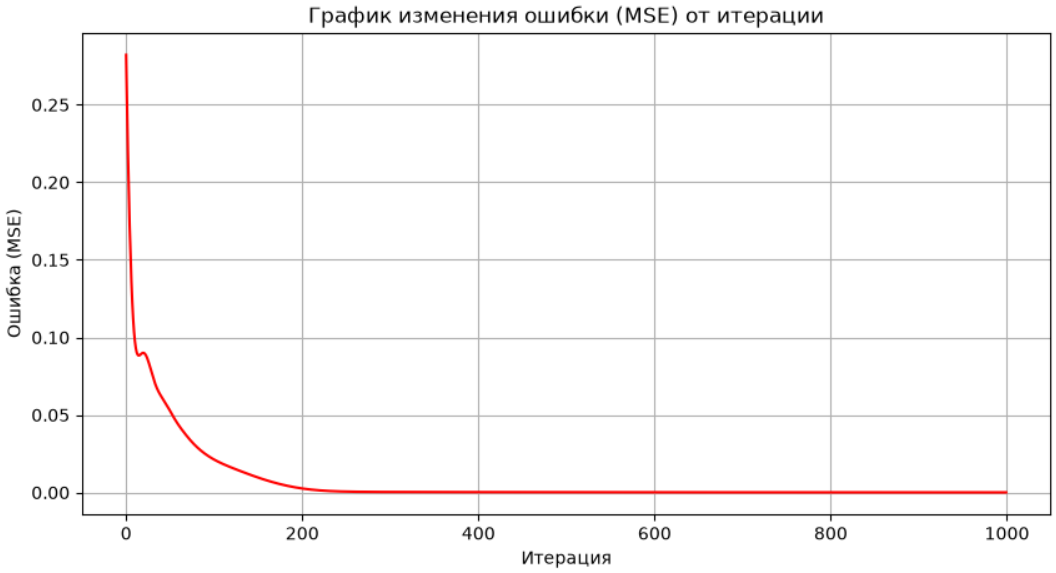
```
plt.show()

# таблицы
def print_table(orig, pred, title):
    df = pd.DataFrame({
        "Эталонное значение": orig,
        "Полученное значение": pred,
        "Отклонение": np.abs(orig - pred)
    })
    print(f"\n--- {title} ---")
    print(df.head(10).round(4).to_string(index=False))

print_table(Y_train_orig, train_pred_orig, "Результаты обучения")
print_table(Y_test_orig, test_pred_orig, "Результаты прогнозирования")
```

## Вывод программы



--- Результаты обучения---

| Эталонное значение | Полученное значение | Отклонение |
|---|---|---|
| 0.1824 | 0.1798 | 0.0026 |
| 0.1713 | 0.1701 | 0.0013 |
| 0.1585 | 0.1585 | 0.0000 |
| 0.1442 | 0.1452 | 0.0010 |
| 0.1284 | 0.1303 | 0.0019 |
| 0.1113 | 0.1137 | 0.0024 |
| 0.0931 | 0.0957 | 0.0026 |
| 0.0740 | 0.0764 | 0.0024 |
| 0.0541 | 0.0561 | 0.0020 |
| 0.0337 | 0.0351 | 0.0014 |

--- Результаты прогнозирования ---

| Эталонное значение | Полученное значение | Отклонение |
|---|---|---|
| 0.0510 | 0.0572 | 0.0062 |
| 0.0709 | 0.0779 | 0.0070 |
| 0.0902 | 0.0974 | 0.0072 |
| 0.1085 | 0.1155 | 0.0069 |
| 0.1258 | 0.1319 | 0.0061 |
| 0.1418 | 0.1467 | 0.0049 |
| 0.1564 | 0.1598 | 0.0034 |
| 0.1694 | 0.1711 | 0.0017 |
| 0.1808 | 0.1806 | 0.0001 |
| 0.1903 | 0.1884 | 0.0019 |

**Вывод**: выполнил моделирование прогнозирующей нелинейной ИНС